



Republic of Iraq
Ministry of Higher Education and
Scientific Research
University of Al Qadisiyah
College of Computer Science &
Information Technology
Information Systems Department



A Template-Based Website Builder

A project
Submitted To Council of the College of Computer Science &
Information Technology, Information Systems Department ,
University of Al Qadisiyah in Partial Fulfilment of the Requirements
for the BSc Degree in Information Systems

By

Mohammad Ali Flayyih

Supervised By

Assistant Prof. Dr. Alaa Abid Muslim Abid Ali

March 2026



(يَرْفَعُ اللَّهُ الَّذِينَ آمَنُوا مِنْكُمْ وَ الَّذِينَ أُوتُوا الْعِلْمَ دَرَجَاتٍ)

[المجادلة: 11]



Dedication

To my beloved family. . .

My Mother

My Father

My Sisters

To my lovely friends

And everyone who cares . . .

Acknowledgments

In preparing this report, I was in contact with many people, researchers, academicians, and practitioners. They have contributed towards my understanding and thought. In particular, I wish to express my sincere appreciation to my main project supervisor, Dr. Alaa Abid Muslim Abid Ali, for encouragement, guidance critics and friendship. I am also very thankful to all my professors for their guidance, advices and motivation. Without their continued support and interest, this project would not have been same as presented here

Librarians at university of Al Qadisiyah. I am also indebted to Information Systems Department faculty; my university also deserve special thanks for their assistance in supplying the relevant literatures.

Supervisor Certification

I certify that the project entitled “A Template-Based Website Builder” was prepared under my supervision at the Information Systems Department / College of Computer Science & Information Technology /University of Al Qadisiyah, as partial fulfillment of the requirements needed for award of the degree of Bachelor in Information Systems.

Signature:

Name: Dr. Alaa Abid Muslim Abid Ali

Date: 15 / 3 / 2026

Email: alaa.abidmuslam@qu.edu.iq

Examination Committee Certification

I hereby certify that this project entitled "A Template-Based Website Builder" Was prepared under my supervision at the Information Systems Department / College of Computer Science & Information Technology /University of Al Qadisiyah, as a partial fulfillment of the requirements for the Bachelor in Information Systems.

(Chairman)

Name:

Date: / / 2026

Signature:

(Member)

Name:

Date: / / 2026

Signature:

Table of

Contents	Page No.
LIST OF TABLES	
LIST OF SCREENSHOT	
LIST OF ABBREVIATIONS	
Abstract	
Chapter One: Introduction	
1.1 Introduction	
1.2 The Problem Statement	
1.3 Research Objectives	
1.4 Project Organization	
Chapter Tow: <u>Background for Project</u>	
2.1 Introduction	
2.2 Languages	
2.3 Frameworks and Libraries	
2.4 Development Tools	
2.5 Database Design	

Chapter Three: Design Implementation	
3.1 Design and Implementation	
3.2 General Structure	
3.3 User Interfaces and Pages	
3.4 State Management and Algorithmic Logic	
Chapter Four: Conclusions and Future Works	
4.1 Conclusions	
4.2 Future Works	
References	
References	

LIST OF TABLES

Table 1	User Collection
Table 2	Project Collection
Table 3	Template Collection

LIST OF SCREENSHOT

Screenshot 1: User Authentication.

Screenshot 2: User Registration Interface.

Screenshot 3: Full Editor Workspace Structure (Sidebar, Canvas, Properties)

Screenshot 4: Preview Mode simulating final website rendering.

Screenshot 5: The internal Template Gallery selection screen.

Screenshot 6: About Page.

LIST OF ABBREVIATIONS

DB	Database
AI	Artificial Intelligent
SW	Software
API	Application Programming Interface
BSON	Binary JavaScript Object Notation
CSS	Cascading Style Sheets
CORS	Cross-Origin Resource Sharing
DB	Database
DOM	Document Object Model
FPS	Frames Per Second
HTML	HyperText Markup Language
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JS	JavaScript
JWT	JSON Web Token
MERN	MongoDB, Express.js, React, Node.js
NoSQL	Not Only SQL (Non-relational database)
REST	Representational State Transfer
SPA	Single Page Application
TWB	Template Website Builder
UI	User Interface
UX	User Experience
VCS	Version Control System
VS Code	Visual Studio Code
WYSIWYG	What You See Is What You Get

Abstract

Establishing a robust and visually compelling web presence has become a fundamental necessity for businesses, creators, and individuals in the contemporary digital era. Despite this, web development remains a polarized field, often divided between professional coding environments that require deep technical expertise and traditional drag-and-drop builders that sacrifice creative freedom and performance for ease of use. This research project introduces the TWB — Template Website Builder, a pioneering platform designed to bridge this gap by offering a high-performance, WYSIWYG (What You See Is What You Get) workspace that prioritizes both user autonomy and technical excellence.

The centerpiece of the TWB project is its innovative Hybrid Positioning Engine. Unlike standard builders that confine users to rigid grid systems, TWB enables a seamless transition between freeform absolute positioning allowing for precise, pixel-perfect design and structured responsive flows that ensure cross-device compatibility. This approach provides the intuitive feel of professional graphic design tools while maintaining the structural integrity and performance of a modern website. Furthermore, the platform addresses the pervasive issue of "code bloat" by generating clean, optimized, and production-ready code in real-time.

Leveraging a robust technical stack comprising React.js, Node.js, and MongoDB, TWB implements advanced features such as an infinite canvas workspace and real-time interactive state previewing. The research concludes that by integrating these professional-grade design capabilities into a no-code environment, TWB successfully democratizes high-end web development. It provides an efficient solution for users seeking to create sophisticated, responsive, and performance-driven websites without the steep learning curve traditionally associated with professional web design tools.

CHAPTER ONE

INTRODUCTION

Chapter One

Introduction

1.1 Introduction

In the contemporary digital era, a web presence is no longer a luxury but a fundamental necessity for businesses, creators, and individuals. However, the process of web development has traditionally been divided into two distinct worlds: professional coding, which offers infinite flexibility but requires deep technical expertise, and traditional drag-and-drop builders, which offer ease of use but often at the cost of creative freedom and code quality.

The Template-Based Website Builder project is made to fill a gap. It gives users a really fast workspace where they can create professional websites without having to write any code. The Template-Based Website Builder uses web technology to provide users a "What You See Is What You Get" experience. This experience feels as easy to use as a design tool. At the same time, the Template-Based Website Builder generates clean and ready-to-use code in the background. The Template-Based Website Builder project is trying to solve a problem with how things are laid out on websites. It does this by using an engine that can handle both freeform and structured layouts. The Template-Based Website Builder is excellent at making websites that look great on all devices.

Chapter One

Introduction

1.2 The Problem Statement

Despite website builders being available, many users still have a hard time creating truly unique digital experiences. The main problems are:

1. **Layout issues:** Most builders only allow you to arrange things in a fixed grid. If you want to move something a little outside of that, it's either not possible or you have to mess around with complicated CSS code.
2. **Hard to learn:** The tools that offer a lot of flexibility often have interfaces that are super overwhelming. It takes months to figure them out, which scares off people who aren't tech-savvy.
3. **Code problems:** Builders that are easy to use often produce code that's messy and slows things down. It's also hard to take that code and use it on your website.
4. **Previews:** Most builders don't let you interact with your design in real time. You can't see how things will look when you hover over them or click on them, which means your design doesn't always match what users will actually see.

Many website builders exist. Users still struggle to create unique digital experiences.

The primary problems users face are:

- **Layout constraints**
- **Learning curves**
- **Code bloat**
- **Static previews**

Chapter One

Introduction

1.3 Research Objectives

The main goal of this research project is to create a website-building platform that lets users make their own websites while following strict rules.

The project has four goals:

1. **Creating a special tool called a Hybrid Positioning Engine:** this tool lets users place elements anywhere on the page, and it also has a grid system to help keep things aligned and functioning well.
2. **Making the website easy to use:** the interface is simple for beginners to navigate. It also has advanced features for people who design websites professionally, ensuring that anyone can use it.
3. **Giving users feedback:** users can see what their website will look like as they are creating it, and they can even test how it will function when others use it.
4. **Creating code that is easy to work with:** when the user is finished building their website, the system converts the design into code that is easy to read and use, making it simple to publish the website online and make changes later.

Chapter One

Introduction

1.4 Project Organisation

This research is organized into four main chapters, designed to guide the reader from the initial concept and technical requirements to the final implementation and future possibilities:

- **Chapter One (Introduction):** Sets the stage by outlining the problem statement, primary research objectives, and the overall scope of the TWB project.
- **Chapter Two (Background):** Provides a comprehensive review of the technical foundation, including the core languages (JavaScript, HTML5, CSS3), the MERN stack (MongoDB, Express, React, Node), development tools, and the detailed database schema.
- **Chapter Three (Methodology and Implementation):** Detailing the system's architecture, the design of the Hybrid Positioning Engine, the internal logic of the Editor and Preview modes, and the core algorithms like Grid Snapping and JWT authentication.
- **Chapter Four (Conclusion and Future Work):** Evaluates the project results against the initial aims, summarizes key achievements, and proposes potential features for future versions.

CHAPTER TWO

Background for Project

Chapter Two

Literature Review

2.1 Introduction

The development of modern web applications has shifted significantly from static HTML pages to dynamic, interactive, and component-based single-page applications (SPAs). Building a complex platform like a Template-Based Website Builder requires a robust architecture capable of handling intensive operations such as real-time Document Object Model (DOM) manipulation, state management, and seamless client-server communication. This chapter provides a general overview of the foundational concepts, programming languages, tools, and database systems utilized to construct this project. It highlights the rationale behind selecting specific technologies over others to achieve high performance and full creative freedom for the end-user.

Chapter Two

Literature Review

2.2 Languages

The project fundamentally relies on the core languages of the web to ensure cross-browser compatibility and high performance.

2.2.1 JavaScript

JavaScript is the primary programming language driving both the client and server sides of the project. Using a unified language stack (often referred to as full-stack JavaScript) significantly streamlines development, data exchange, and system maintenance [1]. It enables complex logic, event handling, and real-time updates within the website builder's intuitive drag-and-drop interface.

2.2.2 HTML5

While JavaScript powers the logic, HyperText Markup Language (HTML5) forms the structural foundation. The website builder utilizes HTML5's extensive array of semantic elements to appropriately structure content, improving the accessibility and machine-readability of the resulting web pages [2].

2.2.3 CSS3

Cascading Style Sheets (CSS) provide the visual foundation. The website builder relies heavily on advanced CSS features-such as Flexbox and CSS Grid-combined with absolute positioning to construct its Hybrid Layout Engine. This allows for responsive design capabilities manipulated directly by the user, facilitating the creation of complex and optimal web layouts [3].

Chapter Two

Literature Review

2.3 Frameworks and Libraries

To accelerate development, ensure high performance, and maintain a scalable architectural foundation, several powerful JavaScript frameworks and libraries were incorporated into the system. These specific frameworks were chosen for their proven capabilities in managing highly intensive user interactions and data-handling tasks.

2.3.1 React.js

React.js is an open-source, component-based JavaScript library developed for building complex and dynamic user interfaces [4]. It is predominantly utilized for constructing Single Page Applications (SPAs) where seamless and high-frequency interactivity is essential. In the context of the Template-Based Website Builder, React plays a pivotal role in managing the complex state of the design canvas.

Several core features make React indispensable for this project:

- **Component-Based Architecture:** React allows the UI to be divided into independent, reusable pieces (e.g., buttons, sidebars, text wrappers, and the canvas itself). This modularity simplifies the codebase and allows each element on the canvas to independently manage its own styles and positions [4].
- **The Virtual DOM and Reconciliation:** Directly manipulating the browser's Document Object Model (DOM) is computationally expensive. When a user drags an element across the builder's canvas, its coordinates (X, Y) update continuously. React addresses this through its "Virtual DOM", an in-memory representation of the actual DOM. React calculates the most efficient way to apply these coordinates (using its reconciliation algorithm) and updates only the specific element completely avoiding full-page re-renders. This drastically minimizes "layout thrashing" and ensures a smooth 60-FPS dragging experience [4].
- **Robust State Management:** Website builders require a single source of truth to track all user changes simultaneously, ranging from selecting an element to modifying its border radius in a properties panel. React provides advanced hooks (such as `useState` and `useEffect`) that make tracking and synchronizing these deep, nested properties efficient and reliable [6].

Chapter Two

Literature Review

2.3.2 Node.js and Express.js

For the backend architecture, the system utilizes Node.js paired with the Express.js framework to create a highly responsive and scalable server environment.

- **Node.js (Event-Driven Architecture):** Node.js is a cross-platform runtime environment that executes JavaScript code server-side. Unlike traditional multi-threaded server models (like Apache), Node.js operates on a single-threaded, non-blocking, event-driven architecture [5]. This design is exceptionally efficient for handling numerous concurrent Input/Output (I/O) operations. In the website builder, when users save projects, the application transmits large JSON trees (representing the layout schema) to the server. Node.js processes these intense data payloads asynchronously, ensuring the server remains responsive without buffering [5].
- **Express.js (Routing and Middleware):** Paired tightly with Node.js, Express.js acts as a minimal yet powerful web framework. It provides a robust set of features for web applications, primarily acting as the routing layer. In this project, Express is utilized to construct highly secure RESTful APIs [1]. It simplifies the implementation of "Middleware" functions—such as parsing incoming JSON requests, establishing secure cross-origin resource sharing (CORS) policies, and authenticating user requests—facilitating secure and rapid communication between the React frontend editor and the database.

Chapter Two

Literature Review

2.4 Development Tools

To facilitate the creation, debugging, and testing of the application, as well as to implement complex interactions safely, several development tools, environments, and specialized libraries were utilized throughout the project lifecycle.

2.4.1 Visual Studio Code

Visual Studio Code (VS Code) is a powerful, lightweight source code editor. It was adopted as the primary Integrated Development Environment (IDE) for its extensive ecosystem of extensions, seamless Git integration, and robust support for the full-stack JavaScript environment [8].

2.4.2 Google Antigravity

Google Antigravity is an advanced agentic AI coding assistant that integrates directly into the development workflow. It was utilized for intelligent code generation, rapid refactoring, debugging complex logic, and assisting in system design, significantly enhancing overall developer productivity [9].

2.4.3 Neovim

Neovim is a highly extensible, terminal-based text editor. It was employed for rapid script editing, server configurations, and efficient shell-based workflows, offering a fast, keyboard-centric environment for quick code modifications [10].

2.4.4 Specifications of the PC used for development:

1. **Processor:** AMD Ryzen™ 5 PRO 5650G @3.9 GHz, Up to @4.4 GHz.
2. **Motherboard:** MSI B450 Gaming Pro Carbon Max WiFi.
3. **Graphics Card:** AMD Radeon™ RX 580 8G Graphics Card .
4. **RAM:** 32GB (3200 GHz).
5. **Disk Storage:** M.2 (256 GB) and SSD (1T).
6. **Operating System:** Arch Linux.

Chapter Two

Literature Review

2.4.5 Web Browsers (Google Chrome & Firefox)

Testing the application and its generated layouts across multiple modern rendering engines is critical, as debugging is deeply context-dependent [11].

Google Chrome: Served as the primary browser for development and live debugging, leveraging the powerful Chrome DevTools to inspect DOM elements, monitor network activity, and analyze React component states.

Mozilla Firefox: Utilized to ensure cross-browser compatibility. Its specialized developer tools provide unique advantages, particularly for deeply inspecting and tweaking complex CSS Grid and Flexbox layouts [11].

2.4.6 React Drag and Drop (React DnD)

A crucial aspect of any modern website builder is the ability to select, move, and place elements on a canvas intuitively. React DnD is a powerful set of React utilities that provides high-level abstractions for handling complex drag-and-drop interfaces. Unlike standard HTML5 drag-and-drop APIs, React DnD allows for isolated, state-driven interactions. It makes it possible to seamlessly transfer data (such as dragged component types and unique IDs) across different sections of the workspace. This library forms the backbone of the Hybrid Positioning Engine, ensuring smooth interactions, accurate dropping calculations, and visual updates for the component layout in real time [6].

2.4.7 Version Control (Git & GitHub)

Robust source code management and collaborative capabilities are fundamental to modern software engineering.

- **Git:** Employed as the primary distributed version control system (VCS). It enables tracking granular changes across the codebase, managing different software versions, and safely reverting to previous states if issues arise.
- **GitHub:** Utilized as the cloud-based hosting platform for the Git repository. It facilitates seamless collaboration, code review via pull requests, issue tracking, and comprehensive project management [12].

Chapter Two

Literature Review

2.5 Database Design

Given the nature of website builders, user projects consist of highly nested, varied data structures (like HTML trees and CSS styles) rather than rigid, uniform data blocks. Therefore, a flexible data storage solution is required.

2.5.1 MongoDB

MongoDB is a leading NoSQL, document-oriented database system that stores data in flexible, JSON-like documents (BSON). This structure perfectly aligns with the website builder's core architecture [7]. In this project, every page or template created by a user is essentially represented as a hierarchical JSON tree containing UI elements, node properties, and styles. Using MongoDB allows the backend to store, query, and update these complex page structures natively and efficiently without the need for strict, predefined relational SQL tables. This ensures that massive templates and deeply customized user projects load quickly and can be saved reliably [7].

Core Collections (Database Tables) The system's database schema is designed around three primary collections to manage users, their ongoing projects, and reusable layout templates:

Table 1: User Collection

(Manages authentication and user profiles).

Field	Data Type (Length)	Content
name	String (min 2)	The user's full name.
email	String	The user's email address (must be unique).
password	String (min 6)	The user's password, securely hashed via bcrypt.
avatar	String	A URL or path to the user's profile picture.

Chapter Two

Literature Review

Table 2: Project Collection

(Acts as the core storage for individual websites created by users).

Field	Data Type (Length)	Content
title	String	The name of the project.
userId	ObjectId	A reference linking the project to its creator (User).
data	Object (JSON)	A flexible JSON object field that stores the entire deeply nested layout structure (UI elements, properties, absolute coordinates) of the website.
isPublished	Boolean	A flag indicating if the website is currently live.

Table 3: Template Collection

(Stores pre-designed, ready-to-use component layouts that users can drop into their projects.)

Field	Data Type (Length)	Content
name	String	The unique identifier or name of the template.
components	Array	An array storing the structural JSON of the template elements.

CHAPTER THREE

Design and Implementation

Chapter Three

Methodology

3.1 Design and Implementation

The methodology adopted for this project revolves around modern web development practices, specifically utilizing a component-based architecture via React.js and a robust backend using the MERN stack (MongoDB, Express.js, React, Node.js). The primary design philosophy of the website builder is to offer a "Hybrid Capabilities" model. Traditional builders generally fall into two categories: strict grid/flow-based (like Webflow) and absolutely free-drag (like Wix). In this system, both approaches are synthesized. Why was this done? To provide advanced users with the structured capabilities of CSS Flexbox and Grid for responsive sections while still retaining the freedom to drag and drop overlapping elements absolutely using X and Y coordinates.

Chapter Three

Methodology

3.2 General Structure

The system's architecture follows a classic Client-Server (Frontend-Backend) model:

- **Frontend (Client):** Built exclusively as a Single Page Application (SPA). The state of the entire project (including multiple pages, nested HTML components, and deeply modified CSS styles) is held in memory using React Hooks (`useState`, `useCallback`, `useEffect`). Global context providers (`AuthContext`, `DndProvider`) wrap the application to manage session states and drag-and-drop contexts efficiently.
- **Backend (Server):** A Node.js environment running Express handles RESTful API requests. The choice to use a NoSQL database (MongoDB) was critical here. Since a user's website layout is essentially a massive, varying JSON tree containing elements and their styles, saving this data in rigid SQL tables would require complex, slow joins. MongoDB naturally accepts these JSON structures directly into the Project collection (`data` field), making load and save operations remarkably fast.

Chapter Three

Methodology

3.2.1 Stateless Authentication (JWT)

To secure the Client-Server communication without degrading the performance of the SPA, Stateless Authentication using JSON Web Tokens (JWT) was implemented. Unlike traditional Session-ID approaches that force the server to repeatedly query the database (increasing server overhead), JWTs carry the user's verified claims (like `userId`) inside a cryptographically signed payload [14]. This ensures data integrity and scalability, allowing the frontend to quickly verify user sessions mathematically without waiting on database latency [15].

Communication between the two layers occurs asynchronously via Axios, with an automated debounced saving mechanism introduced on the frontend to prevent data loss without overwhelming the server with continuous requests on every pixel movement.

Chapter Three

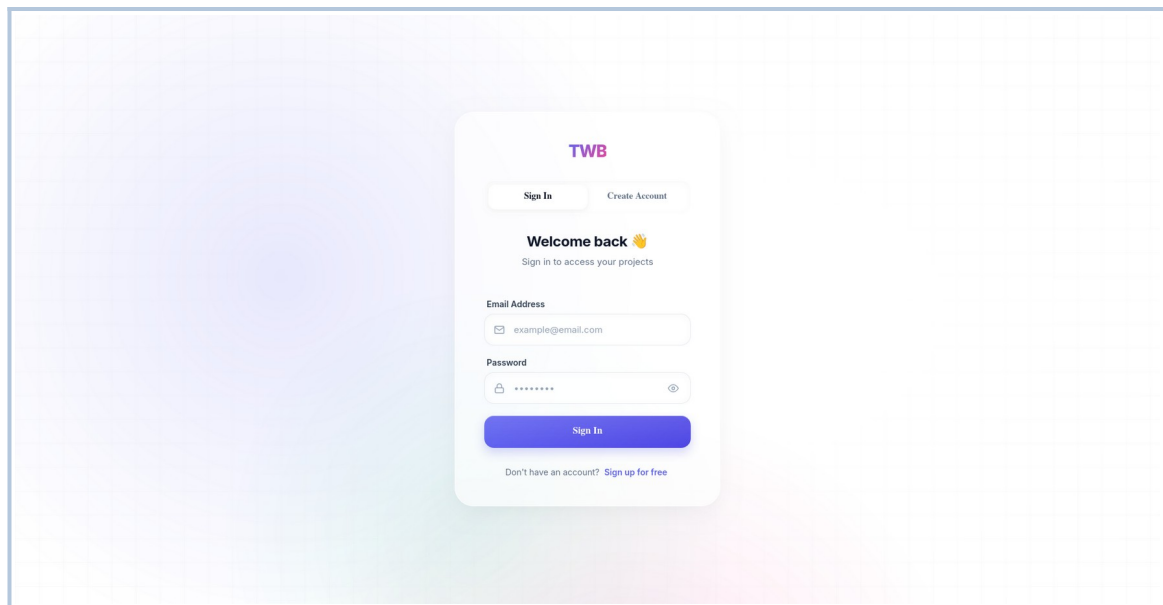
Methodology

3.3 User Interfaces and Pages

The system is heavily modularized, rendering different "pages" or "views" based on user interaction securely bounded by authentication.

3.3.1 User Page (Authentication and Dashboard)

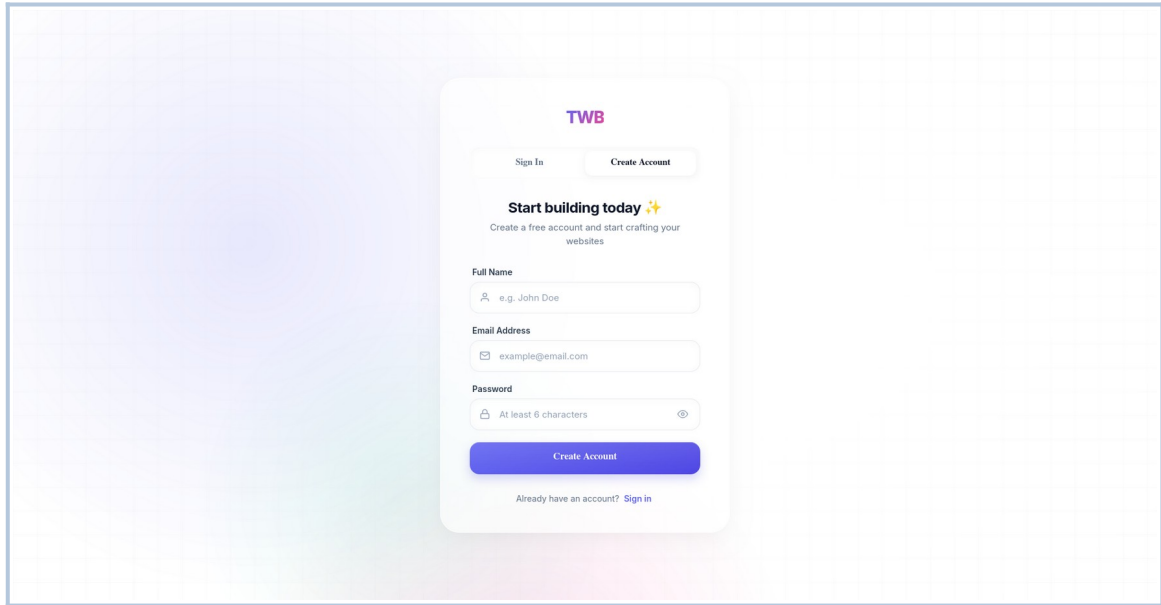
Access to the builder tool requires user identification. The `AuthPage` serves as the gateway, handling user registration and login securely.



Screenshot 1: User Authentication.

Chapter Three

Methodology



Screenshot 2: User Registration Interface.

Design Justification: We implemented an authentication barrier (via `PrivateRoute` bindings) because website data is persistent and personal. By tying a `userId` to a `Project` document in MongoDB, the system ensures data privacy and allows users to seamlessly resume their work from any device. A dropdown User Menu was also designed to give quick access to account details and sign-out functionality.

Chapter Three

Methodology

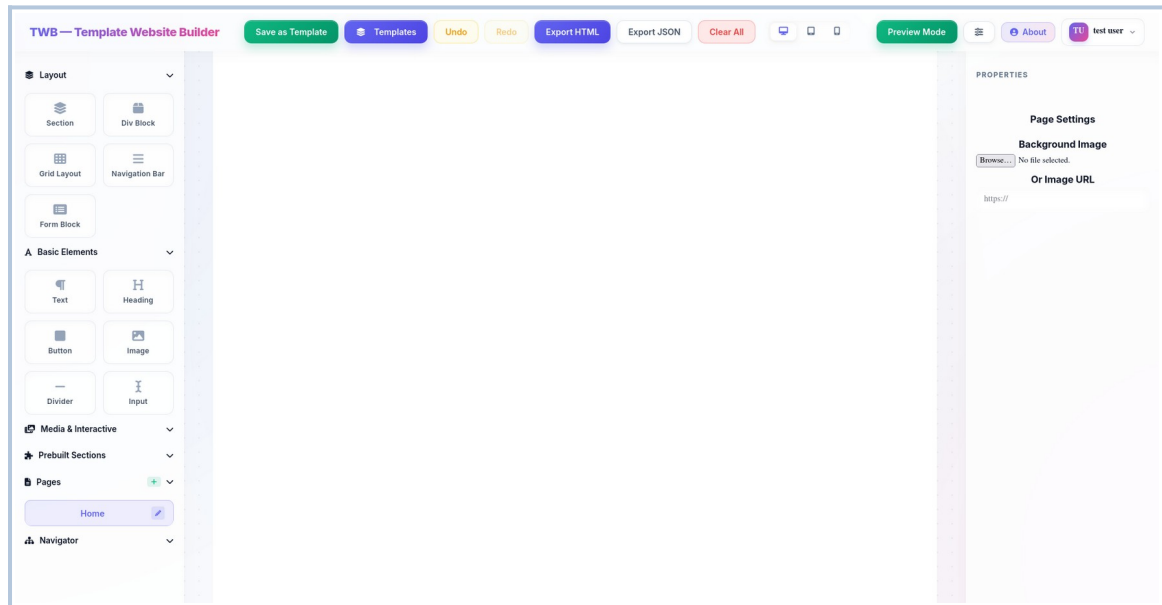
3.3.2 Editor Page

The core workspace (`Editor.js`) is the most computationally heavy page in the system. It is divided into three functional zones:

1. **The Sidebar/Toolbar:** Contains draggable elements categorized logically (Basic, Media, Sections) and a Page Manager. The Page Manager allows users to add, rename, or delete multiple sub-pages (e.g., Home, Contact) within a single project, dynamically updating the React state without refreshing the browser.
2. **The interactive Canvas:** Acts as the drop zone (`useDrop`). It calculates exact mouse coordinates, applying an optional "Grid Snapping" math algorithm to align elements perfectly.
3. **The Properties Panel:** A dynamic panel that reflects the CSS properties of the currently selected element.

Chapter Three

Methodology



Screenshot 3: Full Editor Workspace Structure (Sidebar, Canvas, Properties).

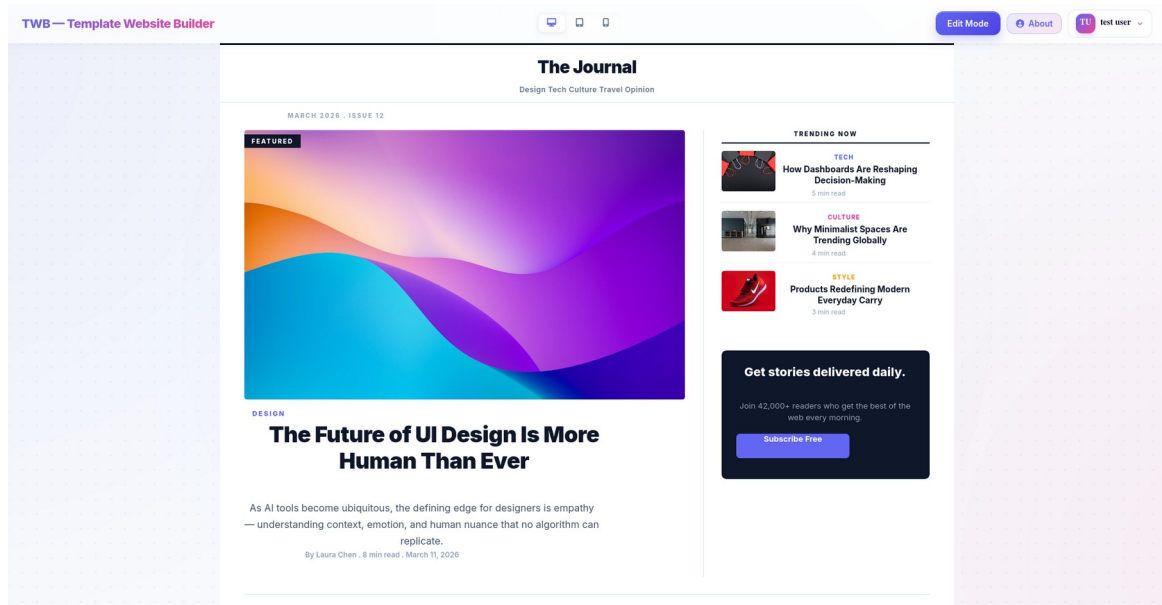
Design Justification: To improve User Experience (UX), a floating toolbar was implemented over selected elements, allowing quick actions like duplication or deletion without having to travel across the screen to the sidebar. Furthermore, an Undo/Redo stack (history and future states) was explicitly built to track changes to the canvas state, forgiving user mistakes instantly.

Chapter Three

Methodology

3.3.3 Preview Page (Mode)

Rather than redirecting the user to a completely separate URL to view their site, a previewMode state toggle is built directly into the Editor.



Screenshot 4: Preview Mode simulating final website rendering.

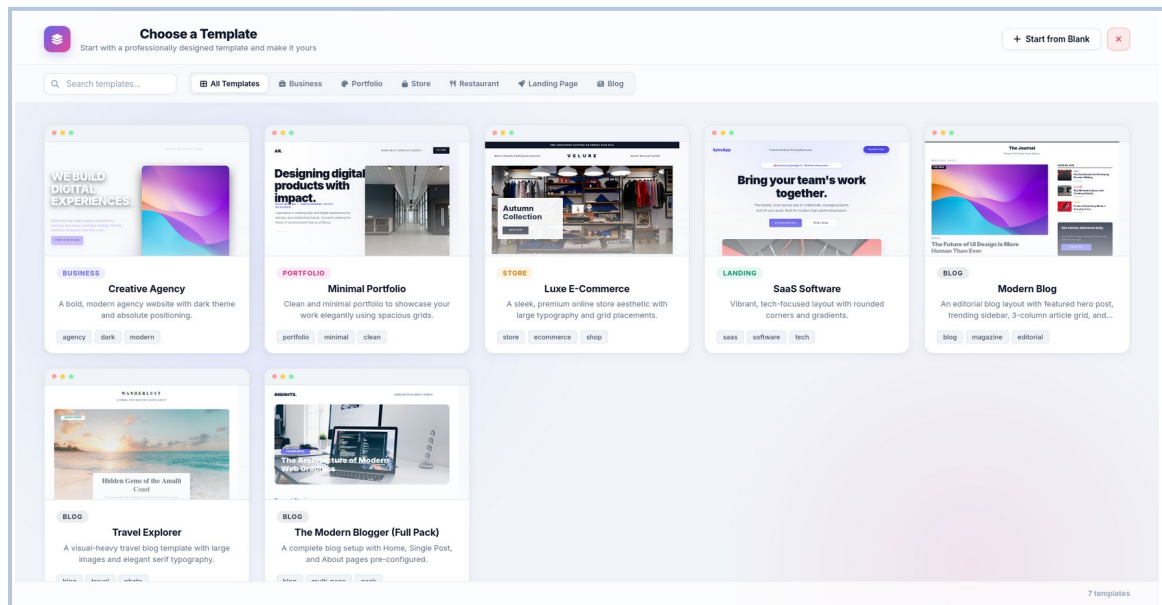
Design Justification: Activating Preview Mode completely hides the sidebar, properties panel, bounding boxes, and grid lines. This decision was made to drastically reduce the feedback loop time. The user can see the exact, final rendered HTML/CSS output instantly, assessing responsive design modes (Desktop, Tablet, Mobile) without the heavy loading times of a traditional external preview link.

Chapter Three

Methodology

3.3.4 Templates Page (Template Gallery)

The Template Gallery is integrated as a modal/overlay page that presents users with pre-designed layouts.



Screenshot 5: The internal Template Gallery selection screen.

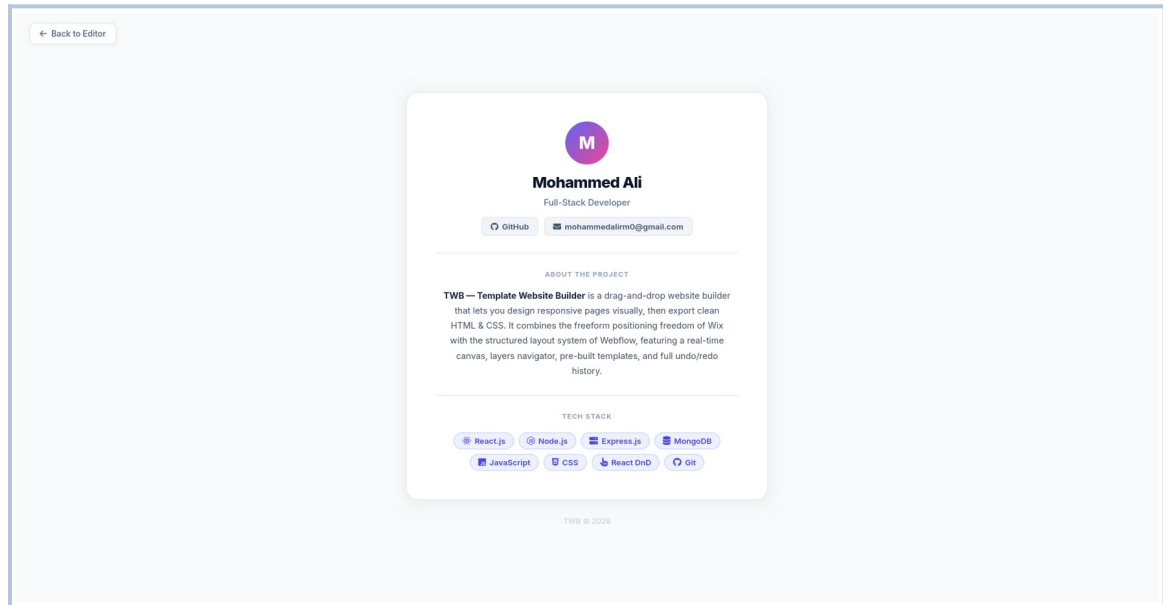
Design Justification: Analyzing user retention in drag-and-drop builders revealed that users often face "blank canvas paralysis." By providing a database-driven `TemplateCollection`, users can inject professionally designed, pre-built component arrays directly into their workspace. This jumpstarts the design process and acts as a learning tool for how complex grids and flexboxes can be constructed.

Chapter Three

Methodology

3.3.5 About Page

The AboutPage serves as informational and instructional documentation accessible directly from the application workspace.



Screenshot 6: About Page.

Design Justification: Instead of hosting documentation externally, placing the About page within the SPA routing ensures that users can quickly learn about system, or project details without disrupting their session flow or losing their unsaved progress.

Chapter Three

Methodology

3.4 State Management and Algorithmic Logic

3.4.1 React State Management

Handling the state of an entire website layout within a browser's memory is computationally expensive. The state of the project (multiple pages, nested HTML components, and deeply modified CSS properties) is held using React Hooks (`useState`, `useCallback`) and Global Contexts. Instead of using heavy global stores like Redux, which increase bundle size and memory footprint, the local component tree manages the updates. The use of the Virtual DOM ensures that when a user drags an element or types in a text box, React only re-renders the specific component (`CanvasItem`) that changed, preserving the 60-FPS smoothness required for a dragging interface [16].

3.4.2 Grid Snapping and Drop Calculations

The drag-and-drop mechanism is not purely visual; it involves an underlying spatial algorithm. When a user drags an element, the `useDrop` hook calculates the exact pointer coordinates (`X`, `Y`) relative to the `Canvas` bounding box. To ensure structural alignment, a Grid Snapping algorithm intercepts these coordinates before rendering:

```
x = Math.round(x / gridSize) * gridSize
```

This mathematical coercion forces randomly dragged elements into a strict, invisible geometric grid (e.g., 10px by 10px), eliminating the misalignment issues commonly found in absolute free-drag editors.

CHAPTER FOUR

Conclusion and Future Work

CHAPTER FOUR

Conclusion and Future Work

4.1 Conclusions

The development of the Template Website Builder (TWB) has successfully demonstrated that the gap between high-level design flexibility and structured web development can be bridged through a robust "Hybrid Capabilities" model. By synthesizing the absolute positioning freedom typical of graphic design tools with the structured flow of modern CSS (Flexbox and Grid), the system provides an intuitive yet powerful workspace for both novice users and professional designers.

Key conclusions drawn from this project include:

- **Efficiency of the Hybrid Engine:** The implementation of a spatial algorithm for grid snapping and coordinate calculation allows for precise layout control while maintaining the responsiveness required for modern web environments. The mathematical coercion logic handles the complexity of absolute-to-relative coordinate conversion seamlessly.
- **Performance-Driven SPA:** Utilizing React.js and its Virtual DOM architecture ensures a high-performance, 60-FPS editing environment. This low-latency feedback loop is essential for a professional-grade drag-and-drop experience.
- **Scalable Data Architecture:** The choice of a NoSQL database (MongoDB) was instrumental in managing the non-linear, hierarchical JSON data structures that represent modern website layouts. This ensures rapid saving and loading of complex projects.
- **Stateless Security:** Implementing JWT-based authentication provided a secure method for managing user sessions without adding overhead to the backend, ensuring a smooth transition between the dashboard and the heavy editor interface.
- **User-Centric Design:** The project successfully integrated a Template Gallery and Preview Mode, significantly reducing the learning curve and helping users overcome the friction of starting from scratch.

In summary, the project has achieved its primary research objectives, delivering a functional, flexible, and scalable platform that simplifies the web design process while maintaining creative control.

CHAPTER FOUR

Conclusion and Future Work

4.2 Future Work

Although the TWB project has established a strong foundation, there are several avenues for further development and research:

- **Full-Site Deployment & Asset Optimization:** Future work should include a "One-Click Publish" feature that optimizes images, minifies CSS/JS, and deploys the project directly to edge hosting services (like Vercel or Netlify).
- **AI-Driven Design Assistance:** Integrating Generative AI models could help users generate content, recommend harmonious color palettes, and even suggest layout modifications based on best practices for specific industries.
- **Real-Time Collaboration:** Expanding the architecture to support WebSockets (via Socket.io) would enable multiple designers to work simultaneously on a single project, fostering collaborative creativity.
- **Component Marketplace & Plugins:** Creating a framework for third-party developers to contribute "Plugins" or custom components would exponentially grow the builder's feature set (e.g., e-commerce integrations, booking systems).
- **Interactive Prototyping:** Adding advanced state management for animations and page transitions would allow the TWB to function as a professional prototyping tool, similar to Framer or Figma.
- **Mobile Design App:** Developing a lightweight mobile version of the dashboard for tracking project analytics and making minor content updates on the go.

The TWB project serves as a starting point for the next generation of intelligent, accessible web building tools, with the potential to democratize high-end web design for everyone.

REFERENCES

- [1] E. Brown, Web Development with Node and Express: Leveraging the JavaScript Stack, 2nd ed. Sebastopol, CA: O'Reilly Media, 2019.
- [2] J. Duckett, HTML and CSS: Design and Build Websites, 1st ed. Indianapolis, IN: Wiley, 2011.
- [3] E. A. Meyer and E. Weyl, CSS: The Definitive Guide, 4th ed. Sebastopol, CA: O'Reilly Media, 2017.
- [4] V. M. Ganeshan, "Web Development Using ReactJS," 2023 5th International Conference on Advances in Computing, Communication Control and Networking (ICAC3N), 2023, pp. 1-5.
- [5] S. Tilkov and S. Vinoski, "Node.js: Using JavaScript to Build High-Performance Network Programs," IEEE Internet Computing, vol. 14, no. 6, pp. 80-83, Nov.-Dec. 2010.
- [6] A. Banks and E. Porcello, Learning React: Modern Patterns for Developing React Apps, 2nd ed. Sebastopol, CA: O'Reilly Media, 2020.
- [7] Y. Li and S. Manoharan, "A performance comparison of SQL and NoSQL databases," 2013 IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM), 2013, pp. 15-19.
- [8] J. Tan, Y. Chen, and S. Jiao, "Visual Studio Code in Introductory Computer Science Course: An Experience Report," ACM, 2023.
- [9] IEEE Computer Society, "The Impact of AI on Productivity and Code in Software Engineering," IEEE, 2024.
- [10] B. Perez et al., "Learning the State Machine Behind a Modal Text Editor: The (Neo)Vim Case Study," IEEE, 2024.
- [11] M. Wang et al., "How Developers Choose Debugging Strategies for Challenging Web Application Defects," ACM, 2024.
- [12] N. Dinh et al., "GitHub Analytics to Support Collaboration in Software Engineering Teams," IEEE, 2024.
- [13] P. Spinellis, "Git," IEEE Software, vol. 29, no. 3, pp. 100-101, 2012.
- [14] J. Nurmi et al., "A JSON Token-Based Authentication and Access Management Schema for Cloud SaaS Applications," IEEE Xplore, 2017.
- [15] R. M. Ahmed and M. Mahmood, "An authentication based scheme for applications using JSON web token," ResearchGate/IEEE, 2019.

[16] V. M. Ganeshan, "Web Development Using ReactJS: Virtual DOM Performance in SPAs," 2023 5th International Conference on Advances in Computing (ICAC3N), 2023.

الملخص

أصبح التواجد القوي والفعال على شبكة الإنترنت ضرورة أساسية لا غنى عنها للشركات والمبدعين والأفراد في العصر الرقمي الحديث. ومع ذلك، لا تزال عملية تطوير المواقع الإلكترونية تمثل تحدياً، حيث تنقسم عادةً بين بيئات البرمجة الاحترافية التي توفر مرونة مطلقة بمتطلبات تقنية معقدة، وبين أدوات بناء المواقع التقليدية التي تضيي بالحرية الإبداعية وجودة الأداء مقابل سهولة الاستخدام. يقدم هذا البحث مشروع أداة إنشاء مواقع الويب الجاهزة "TWB — Template Website Builder"، وهو منصة رائدة صُممت خصيصاً لسد هذه الفجوة من خلال توفير بيئة عمل متطورة تعتمد على مبدأ "ما تراه هو ما تحصل عليه" (WYSIWYG)، والتي تضع استقلالية المصمم والتميز التقني في صدارة أولوياتها.

يتمثل الابتكار الأساسي في مشروع TWB في "محرك التموضع الهجين" (Hybrid Positioning Engine). فخلافاً للمنصات التقليدية التي تحصر المستخدمين في أطر شبكية صلبة، يتيح TWB الانتقال السلس بين التصميم الحر المعتمد على الإحداثيات المطلقة—مما يسمح بدقة متناهية في توزيع العناصر—وبين التدفقات الهيكلية المتجاوبة التي تضمن توافق الموقع مع مختلف أحجام الشاشات. يوفر هذا النهج تجربة تصميم بديهية تحاكي أدوات التصميم الجرافيكي الاحترافية، مع الحفاظ على البنية البرمجية المتينة للموقع. علاوة على ذلك، تعالج المنصة مشكلة "ترهل الكود البرمجي" الشائعة من خلال توليد كود نظيف، محسن، وجاهز للإنتاج الفوري.

بالاعتماد على حزمة تقنية قوية تضم React.js لتطوير الواجهات الديناميكية، و Node.js و MongoDB لإدارة البيانات والبنية التحتية، يطبق المشروع ميزات تنافسية مثل مساحة العمل اللانهائية والمعاينة اللحظية للحالات التفاعلية. يخلص البحث إلى أن دمج هذه القدرات الاحترافية في بيئة تطوير "بدون كود" يسهم بفاعلية في تمكين المستخدمين من بناء تجارب رقمية متطورة، مما يوفر حلاً مثالياً يجمع بين سرعة الإنجاز والجودة العالية دون الحاجة إلى منحنى تعلم برمجي معقد.



وزارة التعليم العالي
والبحوث العلمي
جامعة القادسية
كلية علوم الحاسوب وتكنولوجيا المعلومات
قسم نظم المعلومات الحاسوبية



عنوان المشروع

مشروع مقدم الى قسم نظم المعلومات الحاسوبية كجزء من متطلبات نيل درجة
البكالوريوس في نظم المعلومات الحاسوبية

مقدم من:

محمد علي فليح

بأشراف الأستاذ:

أ.م.د. الاء عبد مسلم عبد علي